

Exploring Factory Pattern

By [Saurabh_Saxena](#), [amrita nevrekar](#), 23 Jun 2009



4.89 (43 votes)

[Download source - 60.75 KB](#)

Table of Contents

- Motivation
- Introduction
- Background
- Definition
- When to go for Factory Pattern
- Using the Code
- Implementations
 - Noob Implementation
 - Advantages
 - Disadvantages
 - Static Factory with Reflection
 - Advantages
 - Disadvantages
 - Self Registration without Reflection
 - Advantages
 - Disadvantages
 - Self Registration with Reflection
 - Advantages
 - Disadvantages
- Difference between Factory pattern and Factory Method pattern
- Advantages of Factory Pattern
- Conclusion
- References

Motivation

During my study of design patterns, I used to find the term *Factory Pattern* very often. I searched the Internet and came across numerous sites. After a lot of search and study, I observed that to find the definition of Factory pattern is no tough deal but it took me quite a while to hit upon a simple explicatory implementation of *Factory pattern* in C#. Most of the examples on the Internet either comprise of *Factory Method Pattern* or *Abstract Factory Pattern*. So I decided to write my own article which targets only *Factory pattern*.

Introduction

An important aspect of software design is the manner in which objects are created, although far more time is often spent considering the object model and object interaction. But if this simple design (of object creation) aspect is ignored, it will adversely impact the entire system. Thus, it is not only important what an object does or what it models, but also in what manner it was created.

I am sure all of us often come across code snippets with `switch` blocks having the 'new' keyword used for instantiating different classes as per client requests. Such a basic mechanism of object creation could result in design problems or added complexity to the design. *Factory pattern* provides an ideal object creation solution suitable to such situations. Moreover, I found that *Factory Pattern* helps to implement the *Open-close Principle*, for more information on OCP, please check my [first article](#). In this article, I will describe the various implementations of *Factory Pattern* and pros and cons of each implementation.

Background

Although *Factory pattern* is one of the most widely used patterns, it is not very familiar to many developers. Perhaps the reason is because it is not included in the *GOF (gang-of-four)* patterns. However, it is worthwhile to note that Factory Pattern is a base for two very popular GOF creational patterns, namely *Factory method* and *Abstract Factory pattern*.

Alright, I think it's been enough of a prologue, let us now excavate into the intricacies of this pattern. I will take you through the Definition of Factory pattern and then describe the various implementation of Factory pattern.

Definition

Well as the name suggests, Factory pattern is a creational design pattern which facilitates:

- Creating objects without exposing the instantiation logic to client
- Referring to the newly created objects through a common interface

I will describe these two points in more detail in the coming sections.

When to go for Factory Pattern

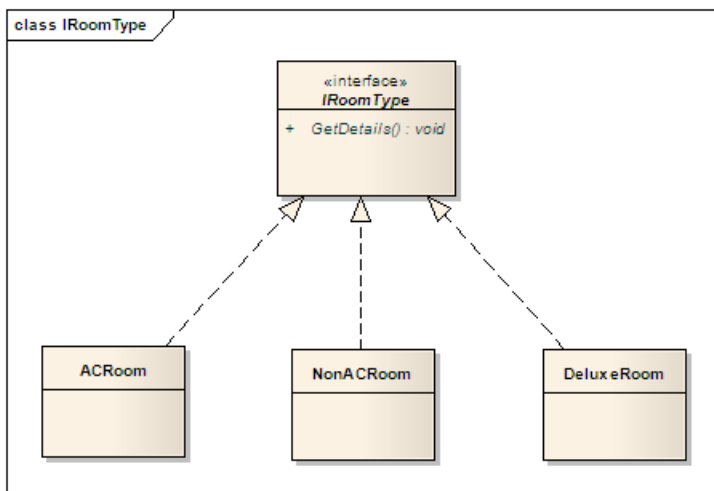
Well, whenever you come across words like **kinds** or **types**, watch out for application of factory pattern!! At the end of the day, it all depends on what the project demands.



Using the Code

There are plenty of places where we can apply the Factory pattern. But at this moment, let's consider a familiar example of a Lodging Inquiry system, wherein a customer may wish to get details of different types of available Rooms. Lodging system provides three different types of Rooms (**DeluxeRoom**, **NonACRoom** or an **ACRoom**). Based on the client choice, my system provides the details of specific room. All very well till this point. But just think about how my system might have been designed here?

With no knowledge of *factory pattern*, a system would have an Interface **IRoomType** which defines the common behavior for all types of room. All the product-classes (namely **ACRoom**, **DeluxeRoom** and **NonACRoom**) implement this interface. Fair enough till this end, now as per the user request, my client application will instantiate the desired class by directly using the **new** keyword.



Typically, the client-code would appear quite like this:

```
private void GetRoomDetails(string roomType)
{
    IRoomType room = null;

    switch (roomType)
    {
        case RoomTypes.AC:
            room = new ACRoom();
            break;

        case RoomTypes.Deluxe:
```

```

        room = new DeluxeRoom();
        break;

    case RoomTypes.NonAC:
        room = new NonACRoom();
        break;
    }
    room.GetDetails();
}

```

So, what is the problem with this type of implementation? Well, before describing the problem, I would like to define one common term "Client application", which I will use quite frequently. Here, I am referring to *any window application that can accept, display and store data is said to be client application*. Let's get back to the real problem.

1. Firstly, we have lots of 'new' keyword scattered in the *client application*. In other words, the *client application* is loaded with a lot of condition based object creational activities which can make the client logic very complicated. What if tomorrow new types of *Rooms* are added in the lodge? Client application code needs to be changed!
2. Client application has to be aware of all the concrete products (*ACRoom*, *NonACRoom* or *DeluxeRoom*) and their instantiation logic. This methodology often introduces a great deal of the inflexibility in the system, as the direct use of a language/run-time provided object instantiation function creates an explicit association between the creator and created classes.
3. The coupling introduced between classes is extremely difficult to overcome if the requirements change (as they always do). Just imagine, there comes a season special offer wherein each *Non-AC room* would be available with a *Deluxe Room* free of cost! In this case, the constructor of *NonACRoom* class takes a reference to *DeluxeRoom* as an argument. Now again, the client application needs to be modified and I am sure after listening to such a drastic change, the developer will get a heart attack.
4. The above mentioned issues might be easy to deal in the case of a small project. But,
 - o What if the client code is huge and too complex to be modified and retested?
 - o And what if this inquiry system is being used by multiple lodges in a city?

In these cases, only God can save the whole application.

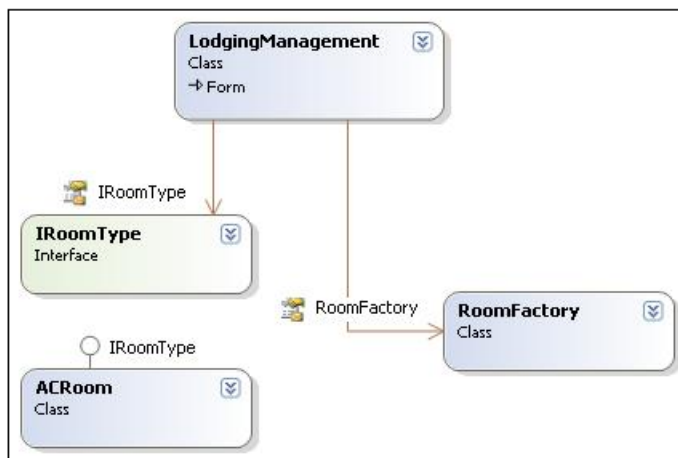
The failure of the above design can be overcome with the aid of *Factory pattern*. *Factory pattern* intends to overcome these drawbacks by delegating the task of object creation to a *factory-class*. The *factory-class* completely abstracts the creation and initialization of the product from the client-application. This indirection enables the client to focus on its discrete role in the application without concerning itself with the details of how the product (*ACRoom*, *NonACRoom* or *DeluxeRoom*) is created. Thus, as the product initialization changes over time or new types of product get added to the system, the client remains unchanged. The flow diagram for the same goes like this:



The *Factory pattern* has evolved over time and majorly there are four different ways of implementing this pattern. Throughout the article, I will take *Lodging Inquiry system* as an example and will describe the pros and cons with each type of implementation. Before going forward, please synchronize with the attached source code.

1. Implementation-1: Procedural Solution/Basic noob implementation also known as parameterized factory.

In noob-implementation, we define a *singleton-factory-class* said to be *RoomFactory* class. *RoomFactory* class is responsible to implement a special-function (say *GetRoomType*). *GetRoomType* function accepts the user requests as a function-parameter and returns specific product-object, by directly using the 'new' keyword. Each product class (say *ACRoom*, *DeluxeRoom* or *NonACRoom*) must have to implement the common interface (*IRoomType*). *IRoomType* interface defines the basic properties for room.



Noob-implementation removes the scattered 'new' from *client code* and gets the same in *RoomFactory* class. This way client-code remains less complicated and unchanged when new products get added to the system. The following code-snippet depicts the implementation of *GetRoomType* method in *RoomFactory* class.

```

internal IRoomType GetRoomType(RoomTypes roomType)

```

```

{
    IRoomType room = null;

    switch (roomType)
    {
        case RoomTypes.AC:
            room = new ACRoom();
            break;

        case RoomTypes.Deluxe:
            room = new DeluxeRoom();
            break;

        case RoomTypes.NonAC:
            room = new NonACRoom();
            break;
    }
    return room;
}

```

In *noob-implementation* of *factory pattern*, the code snippets for client-code will be as follows:

```

private void SubmitButton_Click(object sender, EventArgs e)
{
    RoomTypes rmType = (RoomTypes) "ACRoom";
    IRoomType roomType = RoomFactory.Singleton.GetRoomType(rmType);
    roomType.GetDetails();
}

```

Advantages and disadvantages of this implementation are as follows:

Advantages

- Easy to implement
- Client application code doesn't have to change drastically
- Moreover, the tight coupling between client and product classes is overcome and turned into coupling between factory and product classes. Hence client need not know the instantiation logic of products.

Disadvantages

- If we add any new product (*room*), we need a new *case* statement in *GetRoomType* method of *Factory* class. This violates **open/closed design principle**.
- We can avoid modifying the *Factory* class by using sub classing. But sub classing means replacing all the factory class references everywhere through the code.
- We have *tight coupling* between *Factory* class and products

2. Implementation-2: Registration with reflection

Noob-implementation of *factory pattern* is easy to implement, but leads to a major setback, violation of *open and close principle*. Registration with reflection implementation overcomes the problems of *noob-implementation*. Alright!! Coming back to implementation details, same as *Noob-implementation*, we have product classes (*ACRoom*, *NonACRoom* and *DeluxeRoom*), interface *IRoomType* (defines basic properties of product) and *RoomFactory* class, but internal implementation differs :).

The registration with reflection implementation works on the analogy unique-key association with product classes (*ACRoom*, *NonACRoom* or *DeluxeRoom*). Client sends the corresponding key to get the specific product.

In registration with reflection implementation, *client-application* must have to associate each product class (*DeluxeRoom*, *ACRoom* or *NonACRoom*) with some unique identifier. The client code to register each product is as follows:

```

private void RegisterProduct()
{
    RoomFactory.Singleton.RegisterRoom(RoomTypes.AC, typeof(ACRoom));
    RoomFactory.Singleton.RegisterRoom(RoomTypes.NonAC, typeof(NonACRoom));
    RoomFactory.Singleton.RegisterRoom(RoomTypes.Deluxe, typeof(DeluxeRoom));
}

```

The *RoomFactory* class contains a hash table/dictionary which stores the mapping between product's unique-identifier (key) and type/ product class name (value). *RegisterRoom* method helps to map the unique-identifier with product-class (*ACRoom*, *NonACRoom* or *DeluxeRoom*). The implementation of *RegisterRoom* method in *RoomFactory* class is as follows:

```

public void RegisterRoom(RoomTypes id, Type roomtype)
{
    if (m_RoomMapping.ContainsKey(id) == false)
    {
        m_RoomMapping.Add(id, roomtype);
    }
}

```

The purpose of *GetRoomType* method in *RoomFactory* class is the same but the implementation is slightly different from the *noob-implementation*. Here is the code snippet showing product instantiation logic of *RoomFactory* class:

```
internal IRoomType GetRoomType(RoomTypes roomType)
{
    IRoomType room = null;

    if (m_RoomMapping.ContainsKey(roomType))
    {
        Type className = (Type) m_RoomMapping[roomType];
        room = (IRoomType)Activator.CreateInstance(className);
    }

    return room;
}
```

The big point to notice in this implementation is that, it is the responsibility of client to register each product class in the **RoomFactory** class before using it. The factory class then creates product objects as per client requests not by using “new” keyword but using reflection.

Advantages and disadvantages of this implementation are as follows:

Advantages

- As the factory class remains unchanged even when new products are added, this implementation *doesn't violate the open and closed principle*.
- This implementation provides the loose coupling between **Factory** class and concrete products. This is because product objects are created using reflection so the factory need not know the instantiation details of product class.
- The client doesn't need to be aware of all the available products. Only the required products need to be registered before use. Moreover product creation is abstracted from the client.

Disadvantages

- The client has to register required product classes with the factory, before use.
- Reflection makes system slow.
- Not all the languages support reflection.
- This implementation is more difficult to implement.

3. Implementation-3:Self Registration without reflection

The major problem in the previous implementation is *reflection makes the system slow*. Current implementation (*Self registration without reflection*) is more or less the same as implementation-2 but overcomes the problem of reflection. Same as implementation-2 we have product classes (**ACRoom**, **NonACRoom** or **DeluxeRoom**), **RoomFactory** class and **IRoomType** interface. **IRoomType** interface defines the basic properties for product classes additionally declare **CreateProduct** method. Signature of **IRoomType** interface is as follows:

```
interface IRoomType
{
    void GetDetails();

    IRoomType CreateProduct();
}
```

In *self-registration-without-reflection* implementation, **IRoomType** interface forces each product class (**ACRoom**, **NonACRoom** or **DeluxeRoom**) to implement the **CreateProduct** method. It is crucial to observe the purpose of **CreateProduct** method. **CreateProduct** method returns a new instance of its own type thereby eliminating use of reflection mechanism. The code-snippets of **CreateProduct** method in **NonACRoom** product class is as follows:

```
public IRoomType CreateProduct()
{
    return new NonACRoom();
}
```

In addition to *realizing* the **IRoomType** interface, each *product* class (**DeluxeRoom**, **ACRoom** or **NonACRoom**) must implement a **static** function **RegisterProduct**. **RegisterProduct** method associates the specified key with corresponding product-type using **Factory** class.

```
public static void RegisterProduct(RoomTypes roomType)
{
    RoomFactory.Singleton.RegisterRoom(roomType, new NonACRoom());
}
```

Same as previous implementation, we have **GetRoomType** method in **RoomFactory** class which returns the corresponding product instance based on client request. Implementation of **GetRoomType** method in implementation-3 is as follows:

```
internal IRoomType GetRoomType(RoomTypes roomType)
{
    IRoomType room = null;

    if (m_RoomMapping.ContainsKey(roomType))
    {
        room = (IRoomType)m_RoomMapping[roomType];
    }

    return room.CreateProduct();
}
```

```
}
```

Since the registration process of products (**ACRoom**, **NonACRoom** or **DeluxeRoom**) is contained within the product class itself, this mechanism is termed as **self registration**. Fair enough till this end. Let's come back to the client code, the client registers each product in **RoomFactory** in the following manner:

```
private void RegisterProduct ()
{
    DeluxeRoom.RegisterProduct (RoomTypes.Deluxe);
    ACRoom.RegisterProduct (RoomTypes.AC);
    NonACRoom.RegisterProduct (RoomTypes.NonAC);
}
```

Later, the client code makes use of **GetRoomType** method of **RoomFactory** class to get the corresponding room details.

Advantages

- This implementation gives us the same advantages as in case of Implementation-2.
- Additionally, it eliminates use of reflection. This provides considerable performance gain.

Disadvantages

- The client has to register required product class with the factory before use.
- More difficult to implement

4. Implementation-4:Self Registration with reflection

The problem in the previous two implementations (*implementation-2 and implementation-3*) is product classes (**ACRoom**, **NonACRoom** or **DeluxeRoom**) must have to be registered before using. *Self registration with reflection* implementation removes the pitfall of previous implementation but makes use of reflection. Implementation-4 is almost similar to implementation-2 but the bare difference is the registration is done by **RoomFactory** class itself. In this implementation, each product class has to define one **static** property to returns its unique identifier.

```
public static RoomTypes Key
{
    get
    {
        return RoomTypes.NonAC;
    }
}
```

In order to achieve automatic registration, **RoomFactory** class defines a method, which iterates through all the types within the executing assembly and checks whether the type is a product class (**ACRoom**, **NonACRoom** or **DeluxeRoom**). For example, in this case, we check whether the type implements **IRoomType** interface. For each of these product classes, the **static** unique identifier (**Key**) is retrieved using reflection mechanism. This unique key is registered in hashtable along with corresponding product class type. The code below describes the registration method of **RoomFactory** class:

```
internal void Initialize()
{
    Assembly asm = Assembly.GetExecutingAssembly();

    Type[] allTypes = asm.GetTypes();
    foreach (Type type in allTypes)
    {
        if (type.IsClass && !type.IsInterface)
        {
            Type iRoom = type.GetInterface("IRoomType");

            if (iRoom != null)
            {
                RoomTypes roomType = (RoomTypes)type.GetProperty("key",
                    BindingFlags.Static | BindingFlags.Public).
                    GetValue(null, null);

                if (roomType != null)
                {
                    m_RoomMapping[roomType] = type;
                }
            }
        }
    }
}
```

Definition of **GetRoomType** method in **RoomFactory** class is similar to implementation-2 to retrieve the particular product.

Advantages

- This implementation gives us the same advantages as in case of Implementation-2, additionally client does not need to register each product class with the **RoomFactory** before using it. Registration of product classes will be done

automatically using reflection.

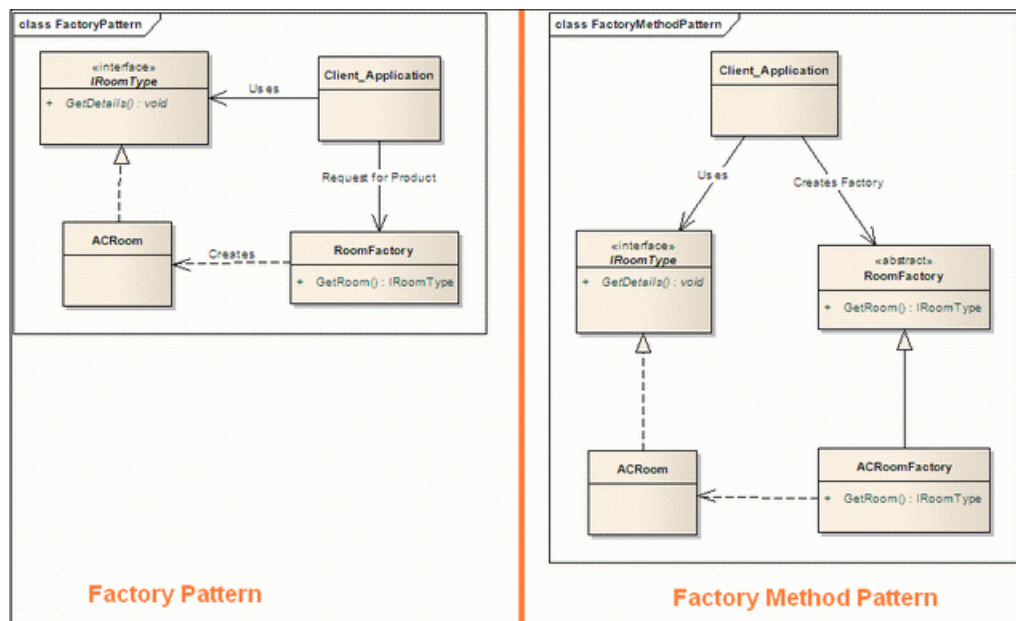
Disadvantages

- Reflection makes system slow.
- Not all the languages support reflection.

Difference between Factory pattern and Factory Method Pattern

While going through this pattern, I often felt that this is more or less the same as factory-method pattern. I was completely baffled, what is the difference between both? Then I arrived at some really good points.

- Factory pattern is the base for **Factory** method and abstract factory pattern.
- Here we have just one factory which is intended to create different types of products whereas in factory method pattern, we need to have one **abstract** class which defines the interfaces for several factories.
- The advantage of factory method pattern over factory pattern is that the client is abstracted from both the type of product and the type of factory used to create the product. Presuming that the product interface is invariant, this enables the factory to create any product type it deems appropriate. Furthermore, presuming that the factory interface is invariant, the entire factory along with the associated products it creates can be replaced in a wholesale fashion. Both of these radical modifications can occur without any changes to the client.



Well, implementation and description of Factory Method pattern is out of the scope of this article. Probably, the next article would focus on the intricacies of **Factory** method pattern.

Advantages of Factory Pattern

One of the eminent facets of a good software design is the principle of loose-coupling or de-coupling, i.e., reducing or removing class dependencies throughout the system. **Factory** class removes the dependency on the specific product classes generated by the **Factory** from the rest of the system. The system is only dependent on the single interface type that is generated by the **Factory**. Loosely coupled or decoupled code is easier to test or reuse since the code has fewer dependencies on other code.

Conclusion

As per the project requirements, the most suitable implementation of Factory pattern can be deployed in the code. The usage of apt implementation will result in considerable fall of code complexity and fair maintenance of code.

License

This article, along with any associated source code and files, is licensed under [The Code Project Open License \(CPOL\)](#)

About the Authors



Saurabh_Saxena

Architect

India 
Member



